

MÓDULO 06: Introducción a Pandas para el Análisis de Datos Científicos

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora 30 minutos

Módulo: 06 - Data Analysis with Pandas: Series, DataFrames, Data Cleaning & Open Formats

1. Objetivos de Aprendizaje

- **Convenciones e Importación:** Comprender la estructura de Pandas y el estándar comunitario `import pandas as pd`.
- **Estructuras de Datos Fundamentales:** Manejar **Series** (unidimensionales) y **DataFrames** (bidimensionales), entendiendo sus índices, ejes y etiquetas.
- **Indexación y Selección:** Dominar las diferencias entre la selección por etiqueta (`.loc[]`) y por posición numérica (`.iloc[]`).
- **Carga de Datasets Reales:** Importar datos científicos desde archivos CSV (`pd.read_csv()`), explorando dimensiones (`.shape`), esquemas (`.info()`) y tipos (`.dtypes`).
- **Limpieza de Datos en Ciencia Abierta:**
 - Tratar valores nulos o faltantes (`dropna()`, `fillna()` con media, mediana o moda, e `isnull().sum()`).
 - Convertir textos a fechas con `pd.to_datetime()` y extraer información temporal (`.dt.year`).
 - Identificar y eliminar registros duplicados (`duplicated()`, `drop_duplicates()`).
 - Comparar DataFrames y detectar discrepancias (`equals()`, `compare()`).
- **Análisis Rápido de Relaciones:** Calcular matrices de correlación lineal (`.corr()`) entre variables numéricas.
- **Exportación a Formatos Abiertos y Publicación:** Exportar DataFrames a tablas LaTeX (`.to_latex()`), CSV (`.to_csv()`), Excel (`.to_excel()`), HTML (`.to_html()`) y JSON (`.to_json()`).

2. Importación de Librerías en Python

Para utilizar Pandas en nuestros scripts o cuadernos de Jupyter, debemos importarlo en el entorno de ejecución.

En Python existen distintas formas de importar librerías según nuestras necesidades:

Forma	Ejemplo	Uso Recomendado
Importar toda la librería	<code>import pandas</code>	Se usa <code>pandas.DataFrame()</code>
Importar con alias (Estándar)	<code>import pandas as pd</code>	Convención universal en Ciencia de Datos. Se usa <code>pd.DataFrame()</code> .
Importar funciones específicas	<code>from pandas import DataFrame, Series</code>	Permite usar <code>DataFrame()</code> directamente. Puede provocar conflictos de nombres.
Importar función con alias	<code>from pandas import DataFrame as DF</code>	Útil únicamente en casos muy específicos de abreviación.



Tip de Ciencia Abierta: Mantener el estándar comunitario `import pandas as pd` garantiza que cualquier investigador en el mundo entienda inmediatamente tu código.

```
In [ ]: # Importamos pandas con la convención estándar pd
import os
import pandas as pd

print(f"Versión de Pandas instalada: {pd.__version__}")
```

```
In [ ]: # Función auxiliar para imprimir líneas separadoras visuales
def linea():
    print("_" * 50)
```

3. Las Series en Pandas: La Unidad Unidimensional Básica

En Pandas, una **Serie** (`pd.Series`) es una **estructura de datos unidimensional** similar a una lista o un vector, pero provista de un **índice asociado** a cada uno de sus elementos.

Características Principales:

- Contiene datos homogéneos (un solo tipo de dato por Serie: enteros, flotantes, cadenas, etc.).
- Asigna un **índice** (etiqueta) a cada elemento para un acceso rápido y seguro.
- Soporta operaciones matemáticas y vectoriales directas sobre sus elementos.

```
In [ ]: # Crear una lista de edades y convertirla a una Serie de Pandas
edades_lista = [16, 23, 45, 18, 21, 45]

linea()
print("Lista original en Python:", edades_lista)
print(f"Tipo original: {type(edades_lista)}")

# Conversión a pd.Series
edades = pd.Series(edades_lista)
linea()
print("Serie de Pandas resultante:")
print(edades)
print(f"Tipo tras convertir: {type(edades)}")
print(f"Segundo elemento (índice posicional 1): {edades[1]}")
linea()
```

Creación de Índices Personalizados en Series

Por defecto, Pandas genera un índice numérico correlativo (0, 1, 2, ...). Sin embargo, en investigación científica es muy habitual utilizar identificadores con significado semántico (nombres de muestras, códigos de sujetos, nombres de autores, etc.).

```
In [ ]: # Crear una Serie especificando el parámetro index
nombres = ["Juan", "Luis", "Carmen", "Gus", "Rocio", "Vladimir"]
edades = pd.Series([16, 23, 45, 18, 21, 45], index=nombres)

linea()
print("Serie con nombres como índice:")
print(edades)
linea()

# Acceso directo por etiqueta
print(f"Edad de Carmen con .loc['Carmen']: {edades.loc['Carmen']}")
print(f"Edad de Carmen con sintaxis edades['Carmen']: {edades['Carmen']}")
linea()
```

```
In [ ]: # Ordenación de valores en una Serie
edades_crecientes = edades.sort_values()
edades_decrecientes = edades.sort_values(ascending=False)

linea()
print("Serie ordenada de menor a mayor edad:")
print(edades_crecientes)
linea()
print("Serie ordenada de mayor a menor edad:")
print(edades_decrecientes)
linea()
```

```
In [ ]: # Filtrado booleano sobre la Serie
# Seleccionar personas con edad estrictamente entre 18 y 40 años
mayores_18 = edades[(edades > 18) & (edades < 40)]
```

```

linea()
print("Personas con edad entre 18 y 40 años:")
print(mayores_18)
linea()

```

```

In [ ]: # Concatenación y modificación de Series
edades2 = pd.Series([30, 28, 35], index=["Ana", "Pedro", "Lucía"])

# Combinamos ambas series con pd.concat()
todos = pd.concat([edades, edades2])

linea()
print("Series unidas con pd.concat():")
print(todos)
linea()

# Añadir una entrada individual
nuevo = pd.Series([52], index=["Ramon"])
todos = pd.concat([todos, nuevo])

print("Serie tras añadir a Ramón:")
print(todos)
linea()

# Eliminar un elemento usando .drop()
todos = todos.drop("Carmen")
print("Serie tras eliminar a Carmen:")
print(todos)
linea()

```

💡 Ojo con la eliminación de elementos (`.drop()`):

En Pandas, la mayoría de los métodos devuelven una **nueva copia** modificada del objeto. Si no asignas el resultado de vuelta a la variable (o no utilizas `inplace=True`), la estructura original **no cambiará**.

```

In [ ]: # Modificación directa con inplace=True
todos.drop("Luis", inplace=True)

linea()
print("Serie tras eliminar a Luis usando inplace=True:")
print(todos)
linea()

```

4. Los DataFrames en Pandas: La Estructura Bidimensional

El **DataFrame** (`pd.DataFrame`) es la estructura de datos bidimensional más potente de Pandas. Es equivalente a una **hoja de cálculo** o una **tabla relacional de base de datos**, compuesta por filas (registros) y columnas (variables).

Características Fundamentales:

- Cada columna es en sí misma una `pd.Series`.
- Cada fila representa una observación completa.
- Las columnas pueden ser de tipos heterogéneos (enteros, números reales, textos, fechas, booleanos).
- Soporta indexación por fila (etiquetas de índice) y por columna (nombres de variable).

```
In [ ]: # Creación de un DataFrame desde un diccionario de Python
dataset_dict = {
    "marcas": ["BMW", "Volvo", "BYD"],
    "modelo": ["X5", "XC90", "Han EV"]
}

linea()
print(f"Tipo del objeto diccionario original: {type(dataset_dict)}")

# Convertimos a DataFrame
df_coches = pd.DataFrame(dataset_dict)

linea()
print(f"Tipo tras convertir a DataFrame: {type(df_coches)}")
print("Contenido del DataFrame resultante:")
print(df_coches)
linea()
```

```
In [ ]: # Configuración de una columna como índice principal mediante .set_index()
df_coches_indexed = df_coches.set_index("marcas")

linea()
print("DataFrame utilizando la columna 'marcas' como índice:")
print(df_coches_indexed)
linea()

# Selección del registro completo por su etiqueta con .loc[]
print("Registro de 'BMW' accediendo con .loc['BMW']:")
print(df_coches_indexed.loc["BMW"])
linea()
```

```
In [ ]: # Diferencias de selección: .loc[] (etiquetas) vs .iloc[] (posiciones numéri
linea()
print("Primer registro por posición con .iloc[0]:")
print(df_coches.iloc[0])
linea()

print("Múltiples registros (filas 0 y 2) con .iloc[[0, 2]]:")
print(df_coches.iloc[[0, 2]])
linea()

print("Inversión del DataFrame mediante slicing .iloc[::-1]:")
```

```
print(df_coches.iloc[::-1])
linea()
```

```
In [ ]: # Conservar la columna original al establecer el índice (drop=False)
df_coches_mantener = df_coches.set_index("marcas", drop=False)

linea()
print("DataFrame con índice en 'marcas' sin eliminar la columna:")
print(df_coches_mantener)
linea()
```

5. Carga e Inspección de Datasets Reales (vivienda.csv)

En ciencia abierta y análisis real, los datos provienen de fuentes externas en formatos abiertos como **CSV** (Valores Separados por Comas).

Cargaremos un dataset con indicadores de vivienda y economía en España descargado de Kaggle.

```
In [ ]: # Carga de datos con pd.read_csv() determinando la ruta de forma flexible
ruta_dataset = "../DATASETS/vivienda.csv" if os.path.exists("../DATASETS/viv

df = pd.read_csv(ruta_dataset)

linea()
print(f"Dataset cargado correctamente desde '{ruta_dataset}'.")
linea()
```

```
In [ ]: # Inspección inicial del tamaño y estructura
filas, columnas = df.shape

linea()
print(f"Dimensiones del dataset: {filas} filas y {columnas} columnas.")
linea()

print("Primeras 5 filas del dataset (.head()):")
print(df.head())
linea()

print("Últimas 3 filas del dataset (.tail(3)):")
print(df.tail(3))
linea()
```

```
In [ ]: # Inspección del esquema técnico y tipos de datos
linea()
print("Información del esquema (.info()):")
df.info()
linea()

print("Tipos de datos por columna (.dtypes):")
```

```
print(df.dtypes)
linea()
```

6. Limpieza de Datos en Ciencia Abierta

La **limpieza de datos** (*Data Cleaning*) es el proceso de detectar y corregir datos incoherentes, incompletos, irrelevantes o corruptos dentro de una tabla de datos.

Anomalías Comunes:

1. **Valores Faltantes (NaN / None)**: Celdas vacías por fallos de recogida.
2. **Tipos de Datos Inadecuados**: Fechas guardadas como cadenas de texto (`object`).
3. **Duplicados**: Filas repetidas que distorsionan el análisis estadístico.
4. **Discrepancias**: Registros divergentes entre distintas versiones del dataset.

6.1. Detección y Tratamiento de Valores Faltantes (NaN)

```
In [ ]: # Forzamos nulos simulados para aprender a gestionarlos
df_simulado = df.copy()

# Insertamos valores vacíos en la columna 'precio'
df_simulado.loc[[99, 150, 300, 415], "precio"] = None

linea()
print(f"Registros originales: {df.shape[0]}")
print(f"Valores nulos en 'precio': {df_simulado['precio'].isnull().sum()}")
linea()
```

```
In [ ]: # Opción A: Eliminar filas con nulos usando dropna()
df_sin_nulos = df_simulado.dropna()

linea()
print(f"Registros tras eliminar nulos con .dropna(): {df_sin_nulos.shape[0]}")
linea()
```

```
In [ ]: # Opción B: Imputación con valores estadísticos (mediana / media) usando fillna()
mediana_precio = df_simulado["precio"].median()

df_imputado = df_simulado.copy()
df_imputado.fillna({"precio": mediana_precio}, inplace=True)

linea()
print(f"Mediana calculada para 'precio': {mediana_precio:.2f} €")
print(f"Valores nulos restantes tras .fillna(): {df_imputado['precio'].isnull().sum()}")
linea()
```

```
In [ ]: # Filtrado de nulos selectivo por columna (parámetro subset)
df_mixto = df.copy()
df_mixto.loc[[100, 200, 300], "num_hipotecas"] = None
```

```
df_mixto.loc[[10, 30, 89], "precio"] = None

# Eliminamos sólo las filas donde 'precio' esté vacío
df_mixto.dropna(subset=["precio"], inplace=True)

# Imputamos 'num_hipotecas' con su mediana
df_mixto.fillna({"num_hipotecas": df_mixto["num_hipotecas"].median()}, inplace=True)

linea()
print(f"Nulos restantes en 'precio': {df_mixto['precio'].isnull().sum()}")
print(f"Nulos restantes en 'num_hipotecas': {df_mixto['num_hipotecas'].isnull().sum()}")
linea()
```

6.2. Conversión de Tipos de Datos y Fechas (pd.to_datetime)

```
In [ ]: # Inspección del formato original de la columna 'year'
primer_valor = df['year'][0]

linea()
print(f"Valor inicial en 'year': '{primer_valor}'")
print(f"Tipo de dato actual: {type(primer_valor)}")
linea()
```

```
In [ ]: # Conversión a datetime con pd.to_datetime()
df['fecha'] = pd.to_datetime(df['year'], format='%Y-%m-%d', errors='coerce')

# Extracción de la componente de año (.dt.year)
df['year_int'] = df['fecha'].dt.year

linea()
print("Conversión a fecha completada:")
print(df[['year', 'fecha', 'year_int']].head())
linea()
```

6.3. Slicing e Indexación (.loc vs .iloc)

```
In [ ]: # Comparación directa de rangos en .loc vs .iloc
linea()
print("Slicing con .loc[10:15] (incluye el índice 15):")
print(df['year_int'].loc[10:15])

linea()
print("Slicing con .iloc[10:15] (excluye el extremo superior 15, muestra 10)")
print(df['year_int'].iloc[10:15])
linea()
```

6.4. Detección y Eliminación de Duplicados

```
In [ ]: # Simulación de un dataset con duplicados
df_duplicado = pd.concat([df, df.iloc[:5]], ignore_index=True)
```

```

linea()
print(f"Filas originales: {df.shape[0]}")
print(f"Filas con duplicados: {df_duplicado.shape[0]}")

# Identificación con .duplicated()
total_duplicados = df_duplicado.duplicated().sum()
print(f"Filas duplicadas detectadas: {total_duplicados}")

# Eliminación con .drop_duplicates()
df_limpio = df_duplicado.drop_duplicates()
linea()
print(f"Filas tras .drop_duplicates(): {df_limpio.shape[0]}")
print(f"¿Restaurado al estado original exacto? {df.equals(df_limpio)}")
linea()

```

6.5. Comparación entre DataFrames (`equals` y `compare`)

```

In [ ]: # Modificación controlada para probar .compare()
df_modificado = df.copy()
df_modificado.loc[100, "precio"] = 9999.0
df_modificado.loc[333, "comunidad"] = "Comunidad_Test"

linea()
print("Diferencias encontradas entre dataframes mediante .compare():")
print(df_modificado.compare(df)[['comunidad', 'precio']])
linea()

```

7. Relaciones entre Variables: Matriz de Correlación (`.corr()`)

En exploración de datos científicos, la **correlación de Pearson** mide la dependencia lineal entre variables numéricas ($[-1, 1]$).

```

In [ ]: import numpy as np

# Filtrar únicamente columnas numéricas
df_numerico = df.select_dtypes(include=[np.number])

# Matriz de correlación redondeada a 2 decimales
matriz_corr = df_numerico.corr().round(2)

linea()
print("Matriz de correlación (primeras 5 columnas):")
print(matriz_corr.iloc[:5, :5])
linea()

```

8. Exportación de Resultados a Formatos Abiertos y Publicación

Uno de los pilares de la Ciencia Abierta es la **reproducibilidad y disponibilidad** de los datos procesados en formatos estándares y libres.

Exportaremos una muestra seleccionada del DataFrame a distintos formatos.

```
In [ ]: # Selección de la muestra a exportar
muestra_exportacion = df.loc[100:108, ["year_int", "comunidad", "precio", "I

linea()
print("Muestra a exportar:")
print(muestra_exportacion)
linea()
```

```
In [ ]: # Exportación a múltiples formatos de ciencia abierta
carpeta_salida = "../DATASETS/" if os.path.exists("../DATASETS") else "DATAS

# 1. LaTeX (.tex) para publicación en artículos científicos
muestra_exportacion.to_latex(f"{carpeta_salida}tabla_seleccionada.tex", inde
print(f"1. Guardado en '{carpeta_salida}tabla_seleccionada.tex'")

# 2. CSV (.csv) formato plano abierto
muestra_exportacion.to_csv(f"{carpeta_salida}tabla_seleccionada.csv", index=
print(f"2. Guardado en '{carpeta_salida}tabla_seleccionada.csv'")

# 3. Excel (.xlsx)
muestra_exportacion.to_excel(f"{carpeta_salida}tabla_seleccionada.xlsx", inc
print(f"3. Guardado en '{carpeta_salida}tabla_seleccionada.xlsx'")

# 4. HTML (.html) para visualización web
muestra_exportacion.to_html(f"{carpeta_salida}tabla_seleccionada.html", inde
print(f"4. Guardado en '{carpeta_salida}tabla_seleccionada.html'")

# 5. JSON (.json) para APIs e interoperabilidad
muestra_exportacion.to_json(f"{carpeta_salida}tabla_seleccionada.json", inde
print(f"5. Guardado en '{carpeta_salida}tabla_seleccionada.json'")
linea()
```

9. Ejercicios Prácticos y Autoevaluación

Ejercicio 1: Filtrado de Comunidades Autónomas

Filtra el DataFrame `df` para obtener únicamente las filas pertenecientes a la comunidad `"andalucia"` con un `precio` mayor a `1500` €. Muestra las primeras 5 filas del resultado.

```
In [ ]: # Solución Ejercicio 1
andalucia_caro = df[(df['comunidad'] == 'andalucia') & (df['precio'] > 1500)]

linea()
print(f"Número de registros que cumplen la condición: {len(andalucia_caro)}")
```

```
print(andalucia_caro[['comunidad', 'year_int', 'precio']].head())
linea()
```

Ejercicio 2: Imputación Estadística por Grupo

Calcula el precio medio (`mean`) de la vivienda por comunidad autónoma.

```
In [ ]: # Solución Ejercicio 2
precio_medio_comunidad = df.groupby('comunidad')['precio'].mean().round(2)

linea()
print("Precio medio de la vivienda por Comunidad Autónoma:")
print(precio_medio_comunidad)
linea()
```

¡Enhorabuena por completar el Módulo 06 sobre Introducción a Pandas! Has adquirido las habilidades fundamentales para cargar, explorar, limpiar y exportar datasets científicos de manera profesional e impecable.



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:

👉 [Cuestionario Módulo 06: Introducción a Pandas](#)
